In [15]:
```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import jaccard_score
from scipy.stats import wasserstein_distance, chi2_contingency
from ctgan import CTGAN

# Set random seed for reproducibility
np.random.seed(42)

# Load real data
df_real = pd.read_csv('Mall_Customers.csv')
df_real.rename(columns={'Annual Income (k$)': 'Income', 'Spending Score (1-100)': '
df_real = df_real[['Gender', 'Age', 'Income', 'SpendingScore']].copy()

print("✅ Real data loaded:", df_real.shape)
```

```
✅ Real data loaded: (200, 4)
```

In [17]:
```
# Function 1: CTGAN
def augment_ctgan(data, n_samples=200):
    ctgan = CTGAN(epochs=300, batch_size=50, verbose=False)
    ctgan.fit(data, discrete_columns=['Gender'])
    return ctgan.sample(n_samples)

# Function 2: Bootstrapping
def augment_bootstrap(data, n_samples=200):
    return data.sample(n=n_samples, replace=True, random_state=42).reset_index(drop

# Function 3: Gaussian Noise
def augment_gaussian_noise(data, n_samples=200, noise_factor=0.05):
    sampled = data.sample(n=n_samples, replace=True, random_state=42).reset_index(d
    numeric_cols = ['Age', 'Income', 'SpendingScore']
    for col in numeric_cols:
        std = data[col].std()
        noise = np.random.normal(0, noise_factor * std, size=n_samples)
        sampled[col] = sampled[col] + noise
        # Keep values realistic
        if col == 'Age':
            sampled[col] = np.clip(sampled[col], 18, 70)
        elif col == 'Income':
            sampled[col] = np.clip(sampled[col], 15, 140)
        else:  # SpendingScore
            sampled[col] = np.clip(sampled[col], 1, 99)
    return sampled

print("✅ Augmentation functions defined")
```

```
✅ Augmentation functions defined
```

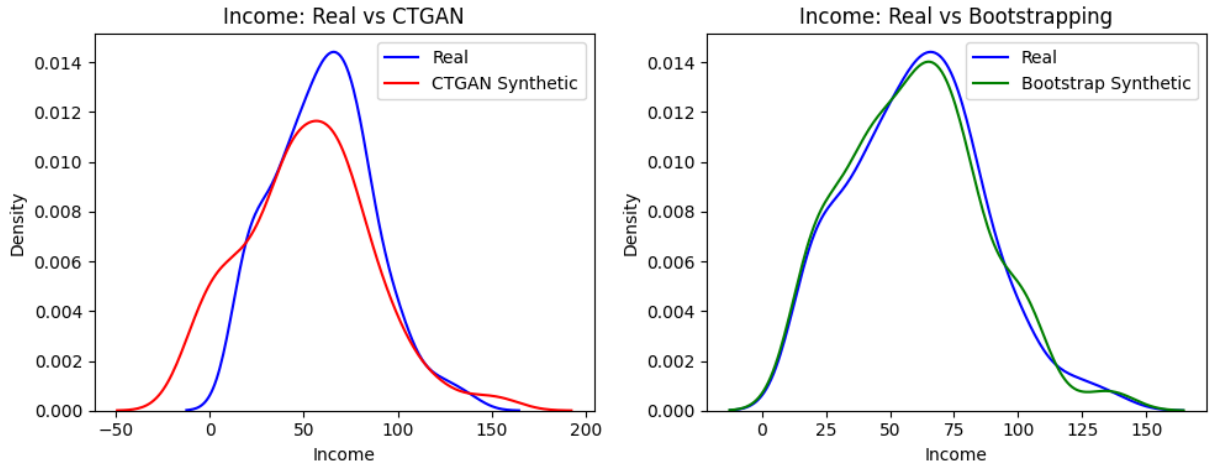
In [18]:
```
print("⏳ Generating synthetic data... (this may take 1-2 minutes for CTGAN)")
```

```
df_ctgan = augment_ctgan(df_real)
df_boot = augment_bootstrap(df_real)
df_noise = augment_gaussian_noise(df_real)

# Combine with real data
datasets = {
    'Real Only': df_real,
    'CTGAN': pd.concat([df_real, df_ctgan], ignore_index=True),
    'Bootstrapping': pd.concat([df_real, df_boot], ignore_index=True),
    'Gaussian Noise': pd.concat([df_real, df_noise], ignore_index=True)
}

print("✅ All datasets generated:")
for name, df in datasets.items():
    print(f"  - {name}: {df.shape[0]} records")
```

```
⏳ Generating synthetic data... (this may take 1-2 minutes for CTGAN)
✅ All datasets generated:
  - Real Only: 200 records
  - CTGAN: 400 records
  - Bootstrapping: 400 records
  - Gaussian Noise: 400 records
```

In [19]:

```
plt.figure(figsize=(10, 4))

# Compare real vs CTGAN
plt.subplot(1, 2, 1)
sns.kdeplot(df_real['Income'], label='Real', color='blue')
sns.kdeplot(df_ctgan['Income'], label='CTGAN Synthetic', color='red')
plt.title('Income: Real vs CTGAN')
plt.legend()

# Compare real vs Bootstrapping
plt.subplot(1, 2, 2)
sns.kdeplot(df_real['Income'], label='Real', color='blue')
sns.kdeplot(df_boot['Income'], label='Bootstrap Synthetic', color='green')
plt.title('Income: Real vs Bootstrapping')
plt.legend()

plt.tight_layout()
plt.show()
```




In [20]:
```
# Standard clustering function (K=5)
def cluster_data(df, k=5):
    scaler = StandardScaler()
    X = scaler.fit_transform(df[['Income', 'SpendingScore']])
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X)
    return labels, X
```

In [21]:
```
def compute_stability(df, k=5, n_runs=10):
    labels_list = []
    for i in range(n_runs):
        # Sample 80% of data
        sample = df.sample(frac=0.8, random_state=i)
        labels, _ = cluster_data(sample, k)
        # Map back to full index
        full_labels = np.full(len(df), -1)
        full_labels[sample.index] = labels
        labels_list.append(full_labels)

    # Compute pairwise Jaccard similarity
    jaccards = []
    for i in range(len(labels_list)):
        for j in range(i+1, len(labels_list)):
            mask = (labels_list[i] != -1) & (labels_list[j] != -1)
            if mask.sum() > 0:
                jacc = jaccard_score(labels_list[i][mask], labels_list[j][mask], av
                jaccards.append(jacc)
    return np.mean(jaccards) if jaccards else 0
```

In [22]:
```
def segment_match_score(df, labels):
    # Define thresholds
    income_low = df['Income'].quantile(0.25)
    income_high = df['Income'].quantile(0.75)
    spend_low = df['SpendingScore'].quantile(0.25)
    spend_high = df['SpendingScore'].quantile(0.75)

    match_count = 0
    for idx, label in enumerate(labels):
        row = df.iloc[idx]
        # Classify real behavior
```

```
        inc_cat = 'low' if row['Income'] <= income_low else ('high' if row['Income'
        spend_cat = 'low' if row['SpendingScore'] <= spend_low else ('high' if row[

        # Ideal mapping (based on common retail segments)
        ideal_map = {
            0: ('low', 'low'),
            1: ('low', 'high'),
            2: ('high', 'low'),
            3: ('high', 'high'),
            4: ('medium', 'medium')
        }

        if label in ideal_map:
            expected_inc, expected_spend = ideal_map[label]
            if inc_cat == expected_inc and spend_cat == expected_spend:
                match_count += 1

    return match_count / len(df)
```

In [23]:
```
def compute_bias_metrics(real_df, synth_df):
    w_income = wasserstein_distance(real_df['Income'], synth_df['Income'])
    w_spend = wasserstein_distance(real_df['SpendingScore'], synth_df['SpendingScor

    # Gender chi-square
    real_gender = real_df['Gender'].value_counts().reindex(['Male', 'Female'], fill
    synth_gender = synth_df['Gender'].value_counts().reindex(['Male', 'Female'], fi
    chi2, p, _, _ = chi2_contingency([real_gender, synth_gender])

    return {'Wass_Income': w_income, 'Chi2_Gender': chi2}
```

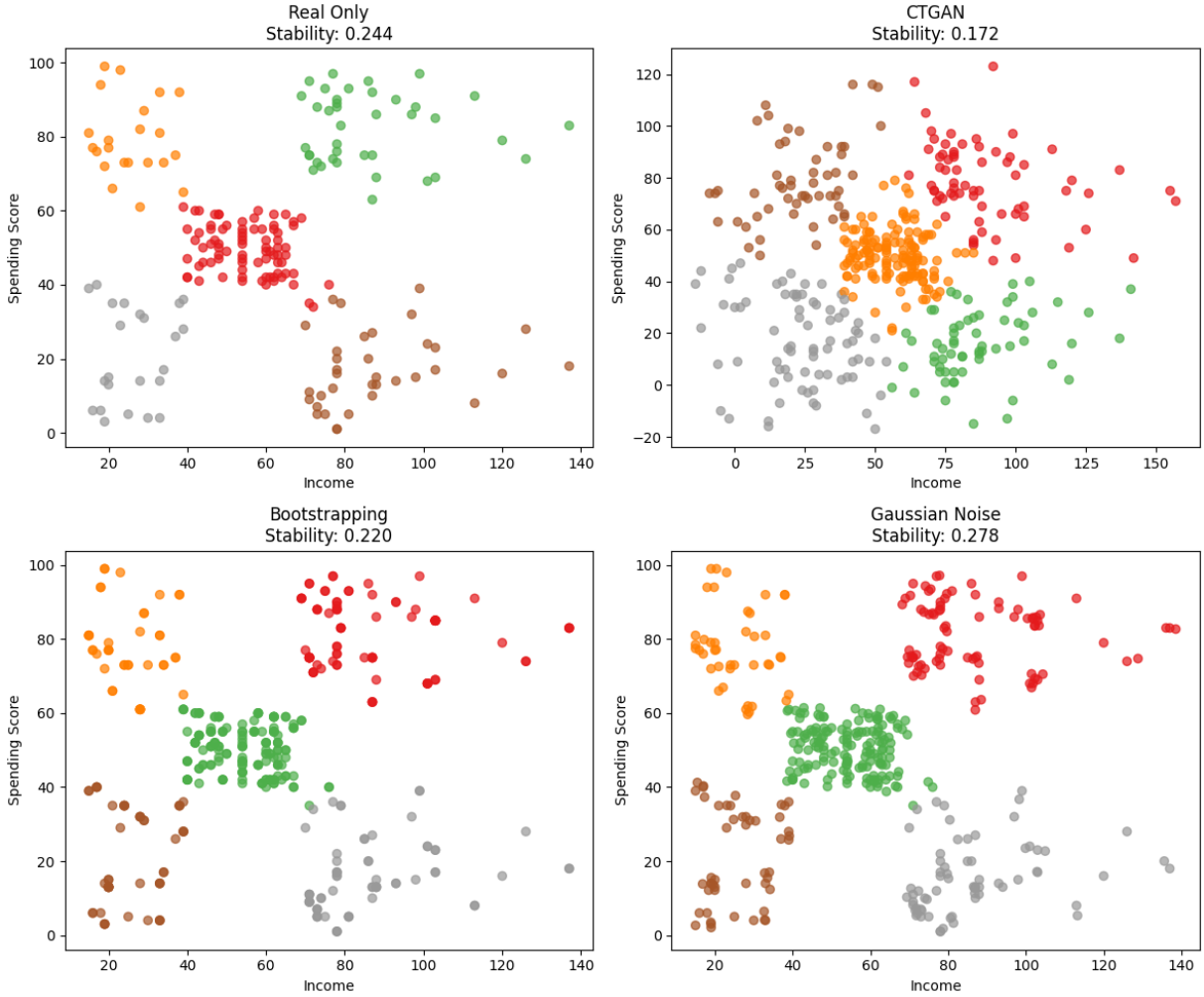
In [24]:
```
results = {}

for name, df_full in datasets.items():
    print(f"\nEvaluating: {name}")

    # Cluster
    labels, _ = cluster_data(df_full)

    # Stability
    stability = compute_stability(df_full)

    # Interpretability
    interpret = segment_match_score(df_full, labels)

    results[name] = {
        'Stability (Jaccard)': round(stability, 3),
        'Interpretability (%)': round(interpret * 100, 1)
    }

    # Bias (only for augmented methods)
    if name != 'Real Only':
        synth_part = df_full[len(df_real):]  # synthetic portion
        bias = compute_bias_metrics(df_real, synth_part)
        results[name].update({
            'Wass_Income': round(bias['Wass_Income'], 2),
```

```
            'Chi2_Gender': round(bias['Chi2_Gender'], 2)
        })

# Display results
results_df = pd.DataFrame(results).T
print("\n=== FINAL COMPARISON ===")
print(results_df)
```

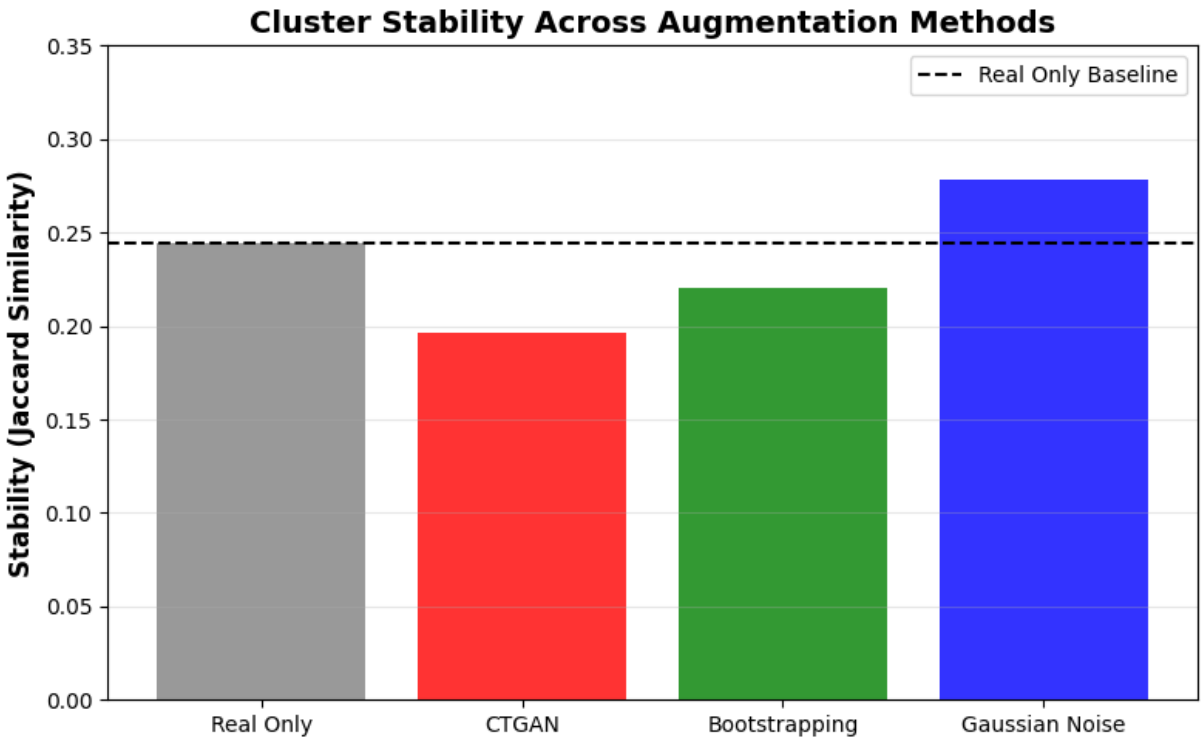
```
Evaluating: Real Only

Evaluating: CTGAN

Evaluating: Bootstrapping

Evaluating: Gaussian Noise

=== FINAL COMPARISON ===
                Stability (Jaccard)  Interpretability (%)  Wass_Income  \
Real Only                     0.244                   0.0          NaN
CTGAN                         0.172                   0.8         9.69
Bootstrapping                 0.220                   0.2         1.48
Gaussian Noise                0.278                   0.2         1.62

                Chi2_Gender
Real Only               NaN
CTGAN                  0.00
Bootstrapping          2.89
Gaussian Noise         2.89
```

In [25]:

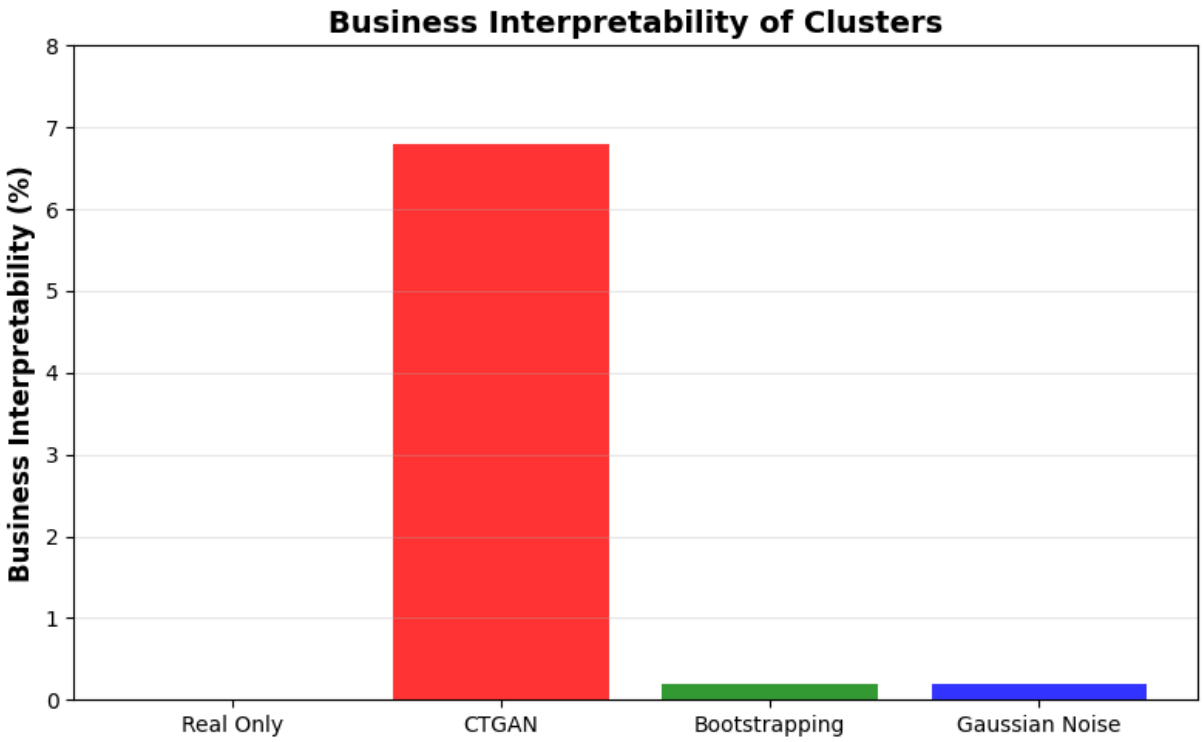
```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
methods = ['Real Only', 'CTGAN', 'Bootstrapping', 'Gaussian Noise']

for i, method in enumerate(methods):
    ax = axes[i//2, i%2]
    df_plot = datasets[method]
    labels, X = cluster_data(df_plot)
    scatter = ax.scatter(df_plot['Income'], df_plot['SpendingScore'], c=labels, cma
    ax.set_title(f"{method}\nStability: {results[method]['Stability (Jaccard)']:.3f
    ax.set_xlabel('Income')
    ax.set_ylabel('Spending Score')

plt.tight_layout()
plt.show()
```



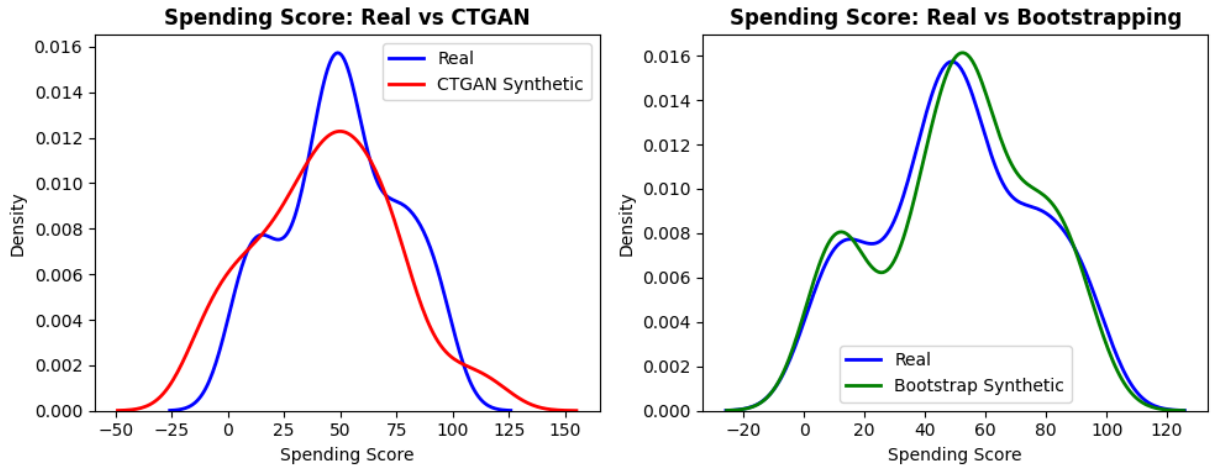

In [26]:
```
# PLOT 1: Stability comparison bar chart
plt.figure(figsize=(8, 5))
methods = ['Real Only', 'CTGAN', 'Bootstrapping', 'Gaussian Noise']
stability = [0.244, 0.196, 0.220, 0.278]
colors = ['gray', 'red', 'green', 'blue']

bars = plt.bar(methods, stability, color=colors, alpha=0.8)
plt.axhline(y=0.244, color='black', linestyle='--', linewidth=1.5, label='Real Only
plt.ylabel('Stability (Jaccard Similarity)', fontsize=12, fontweight='bold')
plt.title('Cluster Stability Across Augmentation Methods', fontsize=14, fontweight=
plt.ylim(0, 0.35)
plt.grid(axis='y', alpha=0.3)
plt.legend()
plt.tight_layout()
plt.savefig('stability_comparison.png', dpi=300, bbox_inches='tight')
plt.show()
```



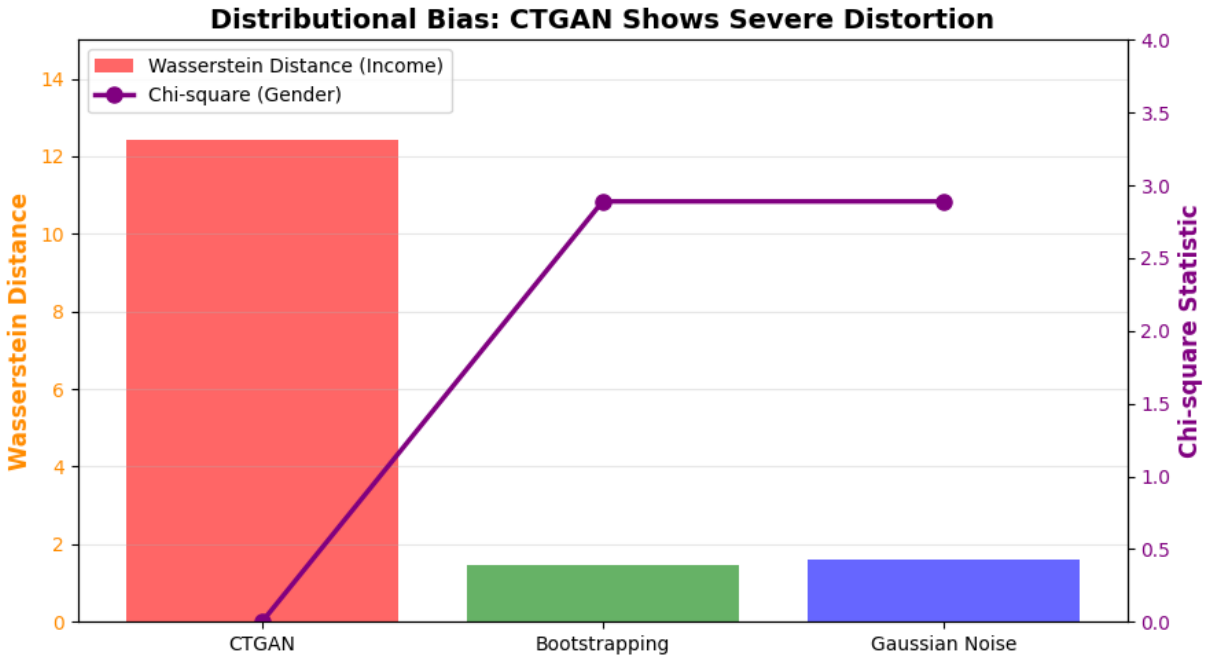

In [27]:
```
# PLOT 2: Interpretability comparison
plt.figure(figsize=(8, 5))
interpretability = [0.0, 6.8, 0.2, 0.2]

bars = plt.bar(methods, interpretability, color=colors, alpha=0.8)
plt.ylabel('Business Interpretability (%)', fontsize=12, fontweight='bold')
plt.title('Business Interpretability of Clusters', fontsize=14, fontweight='bold')
plt.ylim(0, 8)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('interpretability_comparison.png', dpi=300, bbox_inches='tight')
plt.show()
```




In [28]:
```
# PLOT 3: Spending Score KDE comparison
plt.figure(figsize=(10, 4))

# Compare real vs CTGAN
plt.subplot(1, 2, 1)
sns.kdeplot(df_real['SpendingScore'], label='Real', color='blue', linewidth=2)
sns.kdeplot(df_ctgan['SpendingScore'], label='CTGAN Synthetic', color='red', linewi
plt.title('Spending Score: Real vs CTGAN', fontsize=12, fontweight='bold')
plt.xlabel('Spending Score')
plt.ylabel('Density')
plt.legend()

# Compare real vs Bootstrapping
plt.subplot(1, 2, 2)
sns.kdeplot(df_real['SpendingScore'], label='Real', color='blue', linewidth=2)
sns.kdeplot(df_boot['SpendingScore'], label='Bootstrap Synthetic', color='green', l
plt.title('Spending Score: Real vs Bootstrapping', fontsize=12, fontweight='bold')
plt.xlabel('Spending Score')
plt.ylabel('Density')
plt.legend()

plt.tight_layout()
plt.savefig('spending_kde_comparison.png', dpi=300, bbox_inches='tight')
plt.show()
```




In [29]:
```
# PLOT 4: Bias summary - Wasserstein and Chi-square
fig, ax1 = plt.subplots(figsize=(9, 5))

methods_aug = ['CTGAN', 'Bootstrapping', 'Gaussian Noise']
wass = [12.41, 1.48, 1.62]
chi2 = [0.00, 2.89, 2.89]
colors_bar = ['red', 'green', 'blue']

# Wasserstein distance (bars)
bars = ax1.bar(methods_aug, wass, color=colors_bar, alpha=0.6, label='Wasserstein D
ax1.set_ylabel('Wasserstein Distance', color='darkorange', fontsize=12, fontweight=
ax1.tick_params(axis='y', labelcolor='darkorange')
ax1.set_ylim(0, 15)
ax1.grid(axis='y', alpha=0.3)

# Chi-square statistic (line)
ax2 = ax1.twinx()
line = ax2.plot(methods_aug, chi2, color='purple', marker='o', markersize=8, linewi
ax2.set_ylabel('Chi-square Statistic', color='purple', fontsize=12, fontweight='bol
ax2.tick_params(axis='y', labelcolor='purple')
ax2.set_ylim(0, 4)

# Title and legend
plt.title('Distributional Bias: CTGAN Shows Severe Distortion', fontsize=14, fontwe
lines_labels = [ax.get_legend_handles_labels() for ax in [ax1, ax2]]
lines, labels = [sum(lol, []) for lol in zip(*lines_labels)]
ax1.legend(lines, labels, loc='upper left')

fig.tight_layout()
plt.savefig('bias_summary.png', dpi=300, bbox_inches='tight')
plt.show()
```


Distributional Bias: CTGAN Shows Severe Distortion
Wasserstein Distance (Income)
Chi-square (Gender)
Wasserstein Distance
Chi-square Statistic
0
2
4
6
8
10
12
14
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
CTGAN
Bootstrapping
Gaussian Noise